MAPÚA MALAYAN DIGITAL COLLEGE
of Mapúa Malayan Colleges Laguna
In collaboration with
Arizona State University®

This template is an exclusive property of **Mapua-Malayan Digital College** and is protected under **Republic Act No. 8293**, also known as the *Intellectual Property Code of the Philippines* (IP Code). It is provided solely for educational purposes within this course. Students may use this template to complete their tasks but may not **modify**, **distribute**, **sell**, **upload**, or **claim ownership** of the template itself. Such actions constitute copyright infringement under **Sections 172, 177,** and **216** of the IP Code and may result in legal consequences. Unauthorized use beyond this course may result in legal or academic consequences.

Additionally, students must comply with the **Mapua-Malayan Digital College Student Handbook**, particularly with the following provisions:

*** Offenses Related to MMDC IT:**

- Section 6.2 – Unauthorized copying of files
- Section 6.8 – Extraction of protected, copyrighted, and/or confidential information by electronic means using MMDC IT infrastructure

*** Offenses Related to MMDC Admin, IT, and Operations:**

- Section 4.5 – Unauthorized collection or extraction of money, checks, or other instruments of monetary equivalent in connection with matters pertaining to MMDC

Violations of these policies may result in disciplinary actions ranging from **suspension** to **dismissal**, in accordance with the Student Handbook.

For permissions or inquiries, please contact MMDC-ISD at isd@mmdc.mcl.edu.ph.

MAPÚA MALAYAN DIGITAL COLLEGE
of Mapúa Malayan Colleges Laguna
In collaboration with
Arizona State University®

MO-IT153 Information Assurance and Security 2

Implementation Log

Project Name	MotorPH Payroll Management System (Advanced OOP Version)
Group Name/No.	27
System Link	IAS2 Group Folder
Updated Diagram Link	Updated System Architecture Diagram v2

Control #1

Control Name	Parameterized Database Query Execution
Control Type	Technical
Mapped Surface:	
Component	Data Access Layer (`EmployeeDAO.java`, `UserDAO.java`, `PayrollDAO.java`)
Surface Type	Database / SQL Execution Surface
Tool/Library Used:	
Name	Native Java JDBC `java.sql.PreparedStatement`
Version	Java SE (Native)

Notes	Addresses instructor feedback regarding multi-tier architecture separation.
Implementation Summary:	
File(s)/Location(s)	`src/main/java/com/motorph/dao/`
Key change(s) made:	1. Replaced `Statement.executeQuery()` and dynamic string concatenation with `Connection.prepareStatement(query)`. 2. Implemented type-specific parameter binding (`pstmt.setString()`, `pstmt.setInt()`). 3. Refactored DAOs to return native Java objects (e.g., `Optional<User>`) or throw custom `DataAccessException`s, passing execution back to the Logic tier so the Controller properly handles HTTP 400/500 responses.
Date Implemented:	October 16, 2026
Initial Behavior Confirmation (Post-Implementation)	
What we expect to see:	Inputting SQL meta-characters (like `' OR '1'='1' --`) into search fields should be treated as literal strings and return 0 results without crashing.
What we observed:	SQL payloads were neutralized at the database driver level. The search returned empty result sets with no SQL syntax error leaks.
Issues Encountered:	Our baseline DAO methods were improperly formatting and returning HTTP responses directly, violating multi-tier architecture.
Resolution Steps:	Refactored the DAOs to cleanly pass data objects back to the Logic/Service tier, allowing the dedicated validation/controller layers to handle HTTP response formatting.
Next Step:	Execute automated OWASP ZAP Active Scans against the refactored endpoints.
Testing Outcomes	
Test Scenario(s):	SQL Union-Based Exfiltration (`' UNION SELECT null, username, password FROM users --`) in search parameter.
Expected Behavior:	Application processes the payload literally; returns no records.
Observed Behavior:	Database safely parameterized the query; returned empty set.
Evidence Collected:	Evidence_Control1_PreparedStatement_SQLi_Blocked.png
Post-Testing Status:	Verified

	Final Next Step:	Document refactored query methodology in team repository wiki.
	Control #2	
	Control Name	Argon2id Password Hashing & Externalized Configuration Vault
	Control Type	Technical
	Mapped Surface:	
	Component	Authentication Layer & Connection Layer (`DatabaseConfig.java`, `AuthService.java`)
	Surface Type	Authentication & Credential Storage Surface
	Tool/Library Used:	
	Name	`argon2-jvm` & `dotenv-java`
	Version	argon2-jvm:2.11 / dotenv-java:3.0.0
	Notes	Shifted from jBCrypt to Argon2id and refactored config loading based on Milestone 1 instructor feedback.
	Implementation Summary:	
	File(s)/Location(s)	`pom.xml`, `.env`, `DatabaseConfig.java`, `AuthService.java`
	Key change(s) made:	1. Replaced planned bcrypt implementation with Argon2id to meet Advanced requirements. 2. Implemented a static initialization block in `DatabaseConfig.java` to load `Dotenv` only once during application startup, keeping raw DB credentials strictly within the connection layer (removed from the service layer). 3. Expanded the MySQL `password` column to `VARCHAR(255)` to accommodate longer Argon2id hashes. 4. Implemented a dynamic legacy password migration handler inside `AuthService.java` that automatically detects plaintext on login, authenticates it, hashes it to Argon2id, and updates the database record dynamically.
	Date Implemented:	October 16, 2026
	Initial Behavior Confirmation (Post-Implementation)	
	What we expect to see:	App initializes without reading `.env` multiple times. Legacy plaintext logins succeed once, then automatically rehash in the database to a long Argon2id string.

	What we observed:	Database connection succeeded globally on startup. After a seed user logged in, their DB record instantly updated from plaintext to an `$argon2id$v=19$...` hash.
	Issues Encountered:	1) Argon2id hashes exceeded the `VARCHAR(60)` column limit we originally designed for bcrypt. 2) Passing DB keys through the service layer violated separation of concerns.
	Resolution Steps:	1) Executed an `ALTER TABLE users MODIFY COLUMN password VARCHAR(255) NOT NULL;` script. 2) Moved all `Dotenv` variable loading exclusively into a `DatabaseConfig.java` singleton so credentials never touch the service logic.
	Next Step:	Perform negative testing (brute-force attempts) against the new hash structure.
	Testing Outcomes	
	Test Scenario(s):	Log in as the seed user admin, whose passwords is still stored as plaintext, then query the users table to check the stored value.
	Expected Behavior:	The login succeeds and the admin row is automatically upgraded from plaintext to an $argon2id hash. Other users who have not logged in yet stay unchanged.
	Observed Behavior:	After admin logged in, the stored password changed to $argon2id$v=19$m=65536,t=3,p=1$.... hr.andrea and emp.alice remained plaintext because they had not logged in yet.
	Evidence Collected:	Evidence_Control2_Argon2id_Database_Hash.png
	Post-Testing Status:	Verified
	Final Next Step:	Deploy sanitized `.env.example` file to the central repository.